Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс Jetpack Compose



Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 4. Аннотация Inject

21 июня 2021

В этом уроке разбираем использование аннотации Inject

В прошлых уроках мы уже использовали аннотацию Inject. Мы помечали ей объекты (например в Activity), которые даггер должен заинджектить.

Но у этой аннотации есть еще пара применений. Она может быть использована с конструктором объекта и с его методами.

Конструктор

Давайте вспомним обычную схему создания объекта в даггере.

Допустим, нам необходимо получить от компонента объект MainActivityPresenter:

```
@Component(modules = [...])
interface AppComponent {

    fun getMainActivityPresenter(): MainActivityPresenter

}
```

Откуда компонент берет этот объект? Он берет его из модуля.

Мы в модуле создали Provides метод, в котором создаем объект.

```
@Provides
fun provideMainActivityPresenter(): MainActivityPresenter {
    return MainActivityPresenter()
}
```

Компонент вызывает этот метод, получает готовый объект и возвращает его нам.

Но есть и другой способ создания объекта. Мы можем просто добавить аннотацию Inject к конструктору объекта:

```
class MainActivityPresenter @Inject constructor() {
    // ...
}
```

И все. Даггер теперь сам может создать этот объект, используя его конструктор. А Provides метод в модуле больше не нужен, его можно удалить.

Т.е. Inject конструктор - это просто альтернатива Provides методу. Все остальное работает точно также.

Объект созданный этим способом:
- может быть получен от компонента get-методом
- может быть заинджектен компонентом
- может быть использован как аргумент в Provides методе создания другого объекта

Конструктор с аргументами

Но конструктор объекта у нас обычно не пустой. Что если ему на вход нужны другие объекты:

```
class MainActivityPresenter @Inject constructor(
    private val databaseHelper: DatabaseHelper,
    private val networkUtils: NetworkUtils
) {
    // ...
}
```

Это не проблема, если даггер знает, как создать эти объекты. Он создаст их и использует в этом конструкторе.

Причем эти объекты могут создаваться не только в Provides методах модулей:

```
@Provides
fun provideNetworkUtils(): NetworkUtils {
    return NetworkUtils()
}
```

```
@Provides
fun provideDatabaseHelper(): DatabaseHelper {
    return DatabaseHelper()
}
```

но и с помощью Inject конструкторов:

```
class DatabaseHelper @Inject constructor() {
    // ...
}
```

```
class NetworkUtils @Inject constructor() {
    // ...
}
```

Главное, чтобы даггер знал, как создать их.

Т.е. если у вас достаточно простой граф зависимостей, то вы можете обойтись вообще без модулей и Provides методов, используя только Inject конструкторы объектов.

Конструктор с Qualifier аргументами

Если в конструкторе необходимо получить объект с аннотацией Named или Qualifier, то просто указывайте ее перед объектом:

```
class MainActivityPresenter @Inject constructor(
    private val databaseHelper: DatabaseHelper,
    private val networkUtils: NetworkUtils,
    @Dev private val serverApi: ServerApi
) {
    // ...
}
```

Методы

Использование аннотации Inject с методами не так распространено, как с конструктором, но иногда бывает полезным.

Мы можем пометить метод объекта аннотацией Inject, и этот метод будет вызван даггером при создании объекта или инджекте в него. Давайте рассмотрим на примерах.

Создание

Есть объект с Inject конструктором. В нем есть какой-то метод postInit, к которому мы добавляем аннотацию Inject.

```
class MainActivityPresenter @Inject constructor(
    private val databaseHelper: DatabaseHelper,
    private val networkUtils: NetworkUtils
) {

    @Inject
    fun postInit() {
        // ...
    }

}
```

Когда даггер будет создавать этот объект для нас, то сразу после создания он вызовет метод postInit.

Причем он может передать в этот метод объект, который ему доступен для создания:

```
class MainActivityPresenter @Inject constructor(
    private val databaseHelper: DatabaseHelper
) {

    @Inject
    fun postInit(networkUtils: NetworkUtils) {
        // ...
    }

}
```

Важно. Если даггер создает объект с помощью Provides метода из модуля, а не создает сам Inject конструктором, то Inject методы вызваны не будут.

Инджект

Также даггер вызывает Inject методы у объектов, в которые он выполняет инджект. В качестве примера такого объекта мы обычно рассматриваем Activity:

```
class MainActivity : AppCompatActivity() {

    @Inject
    lateinit var mainActivityPresenter: MainActivityPresenter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        (application as App).appComponent.injectMainActivity(this)
    }

    @Inject
    fun postInit(networkUtils: NetworkUtils) {
        // ...
    }

}
```

Когда компонент заинджектит объекты (MainActivityPresenter) в Activity, он вызовет метод postInit.

Модули

Даже несмотря на все удобство Inject конструкторов, остаются случаи, когда нам нужны Provides методы и модули.

1) Создание объекта может быть сложнее, чем просто вызов конструктора.

2) Если объект не наш, то мы не имеем доступа к конструктору этого объекта, чтобы добавить туда Inject аннотацию.

3) Требуются разные варианты создания объектов, обусловленные аннотациями Named и Qualifier.

4) Для инструментальных тестов нам может понадобиться мокать объекты. В этом случае мы просто вместо модулей с реальными объектами, даем компоненту модули с моками.

5) А также есть статья с размышлениями на эту тему.

Назад Вперёд

Комментарии

@Inject — Алексей 21.06.2021 11:29
Здравствуйте. Вы пишете что использование @Inject в конструкторе это аналог провайд метода. Но ведь провайд методы мы пишем в каких-то конкретных модулях каких-то конкретных компонентов. Если существует несколько компонентов то какой из них тогда будет провайдить объект класса, конструктор которого помечен @Inject
Ответить

RE: @Inject — Dmitry Vinogradov 22.06.2021 11:50
Насколько я понимаю, каждый компонент просто сам будет создавать этот объект.

Другое дело, когда включается scope. Тогда уже этот объект будет создаваться в том компоненте, чей scope мы использовали. Из этого компонента объект будет доступен сабкомпонентам.

А остальные компоненты просто не смогут создать такой объект.
Ответить

RE: @Inject — Алексей 22.06.2021 13:28
Ага, понял спасибо. Т.е. если подытожить то получается что объекты, которые провайдятся через методы могут создавать только компоненты к которым подключены соответствующие модули или его сабкомпоненты и компоненты связанные через депрендансис. А объекты классов, конструкторы которых помечены @Inject, могут создавать любые компоненты. Надеюсь правильно понял вас)
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

Комменто

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.